Lab 0: Install and set up Claude Code

Goal

Get Claude Code installed and authenticated on your machine. By the end of this lab you have a working Claude Code environment ready for the rest of the day.

Prerequisites

- A laptop with macOS, Linux, or Windows
- Git installed (git-scm.com)
- An active Claude license (Pro, Max, Team, or Enterprise).

Context

Claude Code is available in several forms. You can run it as a **terminal CLI**, as a **desktop app**, or as an **extension inside VS Code or JetBrains**. All options give you the same agentic capabilities - pick whichever feels most comfortable.

Interface	Best for	How to get it
Terminal CLI	Developers who live in the terminal	Install script (see below)
Desktop app	Visual learners, non-technical participants	Download from claude.com/download
VS Code extension	Anyone already using VS Code	Install from the Marketplace

Not sure which to pick? If you're comfortable in a terminal, go with the CLI - it's the most flexible. If you prefer a graphical interface, the desktop app is a great choice. You can always switch later.

Steps

1. Install Claude Code

Pick **one** of the options below.

Option A: Terminal CLI

macOS / Linux:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows (PowerShell):

```
irm https://claude.ai/install.ps1 | iex
```

Windows (CMD):

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del
install.cmd
```

Homebrew (macOS / Linux):

```
brew install --cask claude-code
```

Option B: Desktop app

Download and install the app from claude.com/download. It runs on macOS and Windows.

Option C: VS Code extension

Install the Claude Code extension from the VS Code Marketplace, or run:

```
code --install-extension anthropic.claude-code
```

2. Log in

Start Claude Code for the first time:

- **Terminal CLI**: open a terminal and run `claude`
- **Desktop app**: open the app
- **VS Code**: open the Claude Code panel (look for it in the sidebar)

You will be prompted to log in. Follow the browser-based authentication flow to connect your Claude account.

Having trouble logging in? Make sure you're using the account that has an active Claude license. Ask your trainer if you need help.

3. Verify the installation

Once logged in, try a quick prompt to make sure everything works:

```
What is 2 + 2?
```

Claude should respond with an answer. If you see a response, you're all set.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

4. Explore the interface

Take a minute to look around. If you're using the terminal CLI, try these commands:

Command	What it does
`/help`	Show all available commands
`/model`	Switch between models (Haiku = cheap, Sonnet = fast, Opus = capable)
`/clear`	Start a fresh conversation
`exit` or Ctrl+D	Exit Claude Code

Check your work

- [ ] Claude Code is installed
- [ ] You are logged in with your Claude account
- [ ] You can send a prompt and receive a response

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 1: Project setup

Goal

Get a correctly scaffolded project with Claude Code ready to go. By the end of this lab, you have a running (empty) project and Claude knows about it.

Prerequisites

- Claude Code installed and authenticated (see Lab 0)
- Git installed
- Your preferred language/framework runtime installed
- Git repository URL for the workshop (your coach will provide this)

Steps

1. Create and scaffold your project

Technical participants: use your preferred tooling to scaffold the project outside of the LLM. This avoids training data cutoff issues with project structure.

Examples:

- `npm create vite@latest my-project`
- `npm init -y && npm install express`
- `dotnet new webapp -o my-project`
- `python -m venv venv && pip install flask`

Initialize git and verify the project runs:

```
cd my-project
git init
# run your app and verify it works
```

Non-technical participants: create an empty directory, start Claude Code, and use this starter prompt:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
I'm not a developer. I want to build a [your idea]. Please choose the simplest
technology, set everything up for me, explain what you're doing in plain
language, and make sure I can run the app with a single command.
```

Ask a coach if you get stuck.

2. Copy the starter-pack into your project

The trainer provides a `starter-pack` folder with pre-built agents, skills, and architecture templates. Copy its contents into your project root.

This gives you:

- `.claude/agents/` - Custom agents for user story mapping and implementation planning
- `.claude/skills/` - Slash commands for story mapping and spec writing
- `.claude/settings.json` - Project settings (including experimental features)
- `docs/architecture/` - Arc42 architecture documentation templates

3. Start Claude Code

```
claude
```

Start Claude from the right folder. Make sure you launch Claude Code from the folder where you unpacked the starter-pack. If you start it from a different directory, the custom agents and skills won't be found and nothing will work as expected.

4. Explore essential commands

Command	What it does
`/model`	Switch between models (Haiku = cheap, Sonnet = fast, Opus = capable)
`/usage`	Show token usage
`/compact`	Compress conversation context (use when context gets long)
`/clear`	Start a fresh conversation
`/exit`	Exit Claude Code
`/help`	Show all available commands

When a session gets too large. Claude Code has a context window limit - the maximum amount of conversation it can hold in memory at once. When your session grows very long you will see a warning that the context is getting full. At that point:

1. Run `/clear` to reset the conversation in the current session, or
2. Exit with `/exit` and relaunch Claude Code to start completely fresh.

Your project files are never affected - only the conversation history is cleared.

5. Generate a CLAUDE.md

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

`CLAUDE.md` is the single most important file for working with Claude Code. It is loaded into Claude's context at the start of every conversation and after every `/clear`. Think of it as your project's permanent briefing to the agent - without it, Claude can read your files but has no idea what matters, what conventions to follow, or how things connect. With a good `CLAUDE.md`, every conversation starts informed instead of clueless.

Run `/init` to let Claude analyze your project structure and generate a first version. Then review what it produces:

- Does it capture the right tech stack and project intent?
- Are there conventions Claude should always follow (naming, folder structure, test commands)?
- Is there anything missing that you'd want every future conversation to know?

Edit the file until it feels like a solid briefing. You'll keep improving it throughout the day.

6. Push to the workshop repository

Your coach has given you a git repository URL. Push your scaffold there so your work is shared and reachable for the rest of the day.

```
git add -A
git commit -m "Initial scaffold"
git remote add origin <coach-provided-repo-url>
git push -u origin main
```

Check your work

- [ ] Project is scaffolded and runs without errors
- [ ] Git is initialized
- [ ] Initial scaffold is pushed to the coach-provided repository
- [ ] Claude Code is running in the project directory
- [ ] CLAUDE.md exists and describes the project

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 2: MCP - Connecting to your team's systems

Goal

Connect Claude to three external capabilities through MCP: an issue tracker, a wiki, and a git repository host. By the end of this lab, Claude can create issues, draft wiki pages, and find code in your real environment - and `CLAUDE.md` holds the project IDs so future prompts don't have to repeat them.

Context

The MCP mechanics are the same across stacks. You point `claude mcp add` at a server, authenticate once when Claude first uses it, and Claude discovers the tools. This lab is intentionally stack-agnostic so it works with whatever combination your team uses.

Before you start: get the project details from your coach

The exercises below need a few identifiers up front. Ask your coach for:

- **Issue tracker** - project key (or board/project name) where you should file your test issue
- **Wiki** - space key (or wiki name) where your test page should live
- **Repositories** - organization or project name to list repos under

Write these in your notes - you'll paste them into the prompts below. They get saved into `CLAUDE.md` later in the lab.

1. Set up the MCP servers

For each capability, register the matching MCP server with project scope (so it's shared via `.mcp.json`) and prefer OAuth. Add only the servers you actually need.

Authentication is lazy: nothing happens at registration time. The first time Claude calls a tool on the server, a browser window opens and you sign in. After that, the session is reused.

Common examples

Atlassian (Jira issues + Confluence wiki)

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

One server covers both:

```
claude mcp add --scope project --transport http atlassian
https://mcp.atlassian.com/v1/mcp/authv2
```

Azure DevOps (work items, wiki, repos)

Stdio server with your organization as input. Add it to `.mcp.json` at the project root:

```
{
  "mcpServers": {
    "ado": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@azure-devops/mcp", "<your-org>"]
    }
  }
}
```

Replace `<your-org>` with the value your coach gave you.

GitHub (issues, repos, PRs)

GitHub needs a Personal Access Token (PAT). The setup is the same for public GitHub.com and GitHub Enterprise (GHE) - only the URLs differ.

1. Create a PAT:
   - **GitHub.com**: `https://github.com/settings/personal-access-tokens`
   - **GHE**: `https://<your-company>.ghe.com/settings/personal-access-tokens` (your coach will give you the exact host)

   [!NOTE] Some organizations enforce SSO. If so, after creating the PAT you'll need to authorize it for your organization ("Configure SSO" next to the token). See Authorizing a personal access token for use with SSO.
2. Add the server to `.mcp.json` at the project root:

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp",
      "headers": { "Authorization": "Bearer ${GITHUB_PAT}" }
    }
  }
}
```

For GHE, use your enterprise host prefixed with `copilot-api.` instead:

```
"url": "https://copilot-api.<your-company>.ghe.com/mcp"
```

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

3. Set the `GITHUB_PAT` environment variable in `.claude/settings.local.json` (this file is git-ignored, so your token stays out of the repo):

```
{
  "env": {
    "GITHUB_PAT": "github_pat_xxxxxxxxxxxxxxxxxxxxxxx"
  }
}
```

[!TIP] Never commit a PAT. Keep it in `.claude/settings.local.json` (git-ignored), not in `.mcp.json`. If a token ever lands in a commit or gets shared, revoke it and generate a new one.

Verify

Run `/mcp` inside Claude Code. Each registered server should appear in the list. Use `/mcp` whenever something fails - it's the first thing to check.

»

2. Why we save what we learn

After each exercise, you'll ask Claude to write the project identifiers it just used into `CLAUDE.md`.

Why bother? Two reasons:

- **Skip the lookup.** Without this, every new conversation has to discover the project key, space ID, or repo URL again - usually with an MCP call or by asking you. With it written down once, future prompts go straight to the action.
- **Stop repeating yourself.** You shouldn't have to type "use the X project in our Atlassian tenant" in every prompt. `CLAUDE.md` is loaded into every conversation, so the agent starts informed.

Each exercise ends with a save step. Don't skip it.

»

3. Create a dummy issue

Ask Claude (fill in the project key from your coach):

```
Use the issue tracker MCP server to create a dummy issue titled "Workshop test -
please ignore" in project <project-key>. Tell me the issue URL when you're done.
```

The first time Claude calls the server, a browser tab opens for you to log in. Complete the OAuth flow and the call resumes.

Then save the details:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
Update CLAUDE.md with the issue tracker details we just used: which MCP server,
the project key, the cloud or organization ID, and anything else you'd need to
file another issue without asking me. Put it under a clear heading.
```

»

4. Create a dummy wiki page

Ask Claude (fill in the space from your coach):

```
Use the wiki MCP server to create a page titled "Workshop test page" in the
<space-key> space. Add a short paragraph of placeholder content so the page
isn't empty. Tell me the page URL when you're done.
```

Then save the details:

```
Update CLAUDE.md with the wiki details: the space key and anything else you'd
need to create another page here without me telling you.
```

»

5. List available repositories

Ask Claude (fill in the org or project from your coach):

```
Use the git repository MCP server to list the repositories under <org-or-
project>. Pick the one matching the project we're working on right now and tell
me its clone URL and default branch.
```

Then save the details:

```
Update CLAUDE.md with the repository details for this project: clone URL,
organization or project name, default branch. Anything else you'd need to find
this repo again without re-running discovery.
```

»

6. Confirm it stuck

Start a fresh conversation with `/clear`, then ask:

```
Without calling any MCP tool, what issue tracker, wiki, and repo are we using
for this project? Where did you find that information?
```

Claude should answer straight from `CLAUDE.md`. If it doesn't, the save step didn't land - go back and fix it.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Check your work

- [ ] At least one MCP server is registered and visible in `/mcp`
- [ ] You completed all three exercises against real systems
- [ ] `CLAUDE.md` contains the issue tracker, wiki, and repo identifiers your coach gave you
- [ ] After `/clear`, Claude still knows the project, space, and repo from `CLAUDE.md` alone

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 3: User story mapping - Let an agent interview you

Goal

Use custom agents to run a user story mapping session and generate spec files. By the end of this lab, you have a set of numbered spec files that describe what to build - generated from a structured interview, not a blank page.

Context

Before building, you need to know what to build. Instead of writing user stories by hand, you'll use two pre-built tools from the starter-pack. First, a custom agent that interviews you about your project and produces a user story map - it runs autonomously with its own model and memory. Then, a skill that turns each story from the map into a detailed spec file - it runs inside your main Claude session rather than as a separate agent. This is your first experience with both agents and skills: two different ways to package reusable logic that extends what Claude can do.

Steps

1. Verify the agents are in place

If you copied the starter-pack in Lab 1, the story mapping and spec agents are already installed. Verify by checking that these files exist in your project:

- `.claude/agents/story-mapping.md`
- `.claude/skills/story-mapping/SKILL.md`
- `.claude/skills/spec/SKILL.md`

If they're missing, go back to Lab 1, step 2 and copy the starter-pack.

2. Generate a user story map

Use the slash command to start the story mapping agent:

```
/story-mapping
```

3. Answer the interview

The agent will interview you **one question at a time** about your project. It will ask about:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

- What you want to build and what makes it exciting
- Who the primary user is and what they're trying to do
- The main activities the user performs
- The stories needed for each activity

Answer in your own words. The interview is quick - about 5-8 questions. The goal is a story map that's ambitious enough to be impressive but broken into stories small enough to actually build.

Not a developer? This is where you shine. You know what you want to build better than anyone. The interview is about goals and users, not technology. Just describe your vision.

4. Review the story map

The agent produces a numbered user story map organized by user activities. Look through it:

- Does it capture the core of what you described?
- Is the goal clear?
- Do the numbers and grouping make sense?
- Is it ambitious enough to be a cool demo?

Edit the story map if needed, or ask Claude to adjust it. Then move on - you can always add stories later.

5. Publish the story map to the wiki

Once the story map markdown is in your project, start a fresh conversation with `/clear`. Then ask:

```
Put the story map on the wiki.
```

Notice how short the prompt is. Claude finds the story map in your project and reads the wiki space out of `CLAUDE.md` (saved in Lab 2), then calls the wiki MCP tools without you specifying any IDs. This is the payoff for the work you did in Lab 2.

Open the wiki in your browser and confirm the page is there.

6. Generate spec files

Pick a story from your story map and run the spec agent:

```
/spec <story-number-or-name>
```

The agent will interview you about what the story should look like - what it does, who it's for, how it behaves - and then generate a spec file in `docs/specs/`. The spec file

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

number matches the story number from the map, so you can always trace a spec back to its story.

Short on time? You can skip the interview by giving the agent everything upfront in a single command - target user, goal, key behaviors, what's out of scope. The agent will produce a draft spec directly.

Repeat for each story you want to spec out. You don't have to spec everything now - just the ones you plan to build first.

7. Review and prioritize

The agent will ask you to review each spec before saving. Look through the generated spec files:

- Are the stories scoped small enough to build one at a time?
- Is anything missing or surprising?

Decide which specs to build first. Put the most important or foundational stories at the top - you'll implement these in the next lab.

8. Sync the specs to the issue tracker

Start another fresh conversation with `/clear`. Then ask:

```
Sync the user stories in the specs to the issue tracker.
```

Same pattern as before. Claude reads the spec files from your project, picks up the issue tracker project key out of `CLAUDE.md`, and creates one issue per spec - without you specifying any IDs.

Open the issue tracker in your browser and verify the issues appear.

Check your work

- [ ] Story mapping agent ran successfully and produced a story map
- [ ] Story map is published as a wiki page (visible in your browser)
- [ ] Spec skill generated at least 2-3 spec files
- [ ] Spec file numbers match the story numbers in the map
- [ ] Each spec has a matching issue in the issue tracker (visible in your browser)
- [ ] You know which story to build first

Reflection

- How was the experience of being interviewed by an agent?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

- Did the story map and spec files capture your intent accurately?
- How does this compare to writing user stories by hand?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 4: Building features - From spec to working code

Goal

Implement features from your spec files by directing Claude. By the end of this lab, you have 2-3 working features and a feel for the rhythm of agentic development.

Context

You have spec files from the story mapping session. Now you turn them into working code - not by writing it yourself, but by giving Claude a spec and letting it build.

Steps

1. Implement your first feature

Pick the highest-priority spec file and give it to Claude:

```
Implement spec <spec number here>
```

Watch how Claude works:

- It reads the spec, the CLAUDE.md, and the existing codebase
- It creates files, installs dependencies if needed, and writes code
- It asks for permission before running shell commands

2. Review and iterate

Check the result:

- Does it match the spec?
- Does it work in the browser?
- Is anything missing or wrong?

If something isn't right, describe the problem and let Claude fix it. Don't fix it yourself - that's the workflow we're practicing.

3. Implement 1-2 more features

Repeat the process for your next specs. With each feature, notice the rhythm:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

- Give Claude a spec
- Let it work
- Review
- Iterate if needed
- Move on

Not a developer? You're doing the same thing as everyone else: describing what you want and reviewing what you get. If something doesn't look right, describe what's wrong in plain language. "The button should be on the right side" or "I expected the list to be sorted by date" - Claude understands.

4. Update the architecture docs

As you add features, the architecture docs in `docs/architecture/` are probably drifting out of date. Claude is building code, not maintaining documentation - so unless you ask, the ARC42 files won't reflect what you just built.

You can fix this with a custom skill. Ask Claude to generate one for you:

```
Create a skill in .claude/skills/update-arc42/SKILL.md that updates the ARC42
architecture documentation in docs/architecture/ to reflect the current state of
the codebase. It should read the existing docs, inspect the code, and update
each ARC42 markdown file in parallel using parallel subagents.
```

Once the file exists, run it any time with:

```
/update-arc42
```

The skill reads the ARC42 files, inspects what was built, and updates the relevant sections - keeping the docs in sync without you having to do it manually.

You'll do more of this in Lab 5 - generating custom agents for specific repeatable jobs. This is a preview of that pattern.

5. Push your work so far

Push everything you've built to the coach-provided repository:

```
git add -A
git commit -m "Implement initial features"
git push
```

If you've been pushing along the way, give it one more push so the latest features and updated docs are on the remote.

Check your work

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

- [ ] At least 2 features are implemented and working
- [ ] Each feature matches its spec
- [ ] The app runs and features work together
- [ ] The ARC42 architecture docs reflect the current state of the project
- [ ] Your work is pushed to the coach-provided repository

Tip

If something goes wrong, don't manually fix the code. Tell Claude what's broken and let it fix it.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 5: Subagents - A custom code review agent

Goal

Build one feature on a branch, raise a pull request, and let a custom code review subagent leave review comments through the git MCP. By the end of this lab, you've shipped a feature through a real PR review flow - with the review automated by your own agent.

Context

So far, Claude has done everything in one conversation thread. That works fine for small tasks, but it has downsides: messy exploration clutters your context, and Claude reviewing its own work in the same conversation is biased toward what it just built. Subagents fix both: they have their own isolated context, and you can give them a specific job that they do well repeatedly.

In this lab you write a code review agent. To make the review meaningful, you set it loose on a real change: a feature you build on a branch and open as a pull request.

Steps

1. Pick a user story to implement

Pick a story from your story map that you haven't built yet - small enough to fit into one PR, big enough to be worth reviewing.

2. Create a feature branch

From your project root:

```
git checkout -b feature/<short-name>
```

3. Implement the feature

Ask Claude to implement the spec for the story you picked:

```
Implement spec <spec number here>.
```

Review the result and iterate until it works. Same rhythm as Lab 4.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

4. Push the branch

```
git add -A
git commit -m "Implement <feature name>"
git push -u origin feature/<short-name>
```

5. Open a pull request

Ask Claude:

```
Create a pull request for this branch.
```

Claude uses the git MCP server you configured in Lab 2 - it already knows the repository details from `CLAUDE.md`, so you don't have to repeat them. Confirm the PR shows up in your browser.

6. Create the code review agent

Now build the subagent that will review the PR. Ask Claude:

```
Create a code review agent in .claude/agents/reviewer.md. It should check code
against common issues like error handling, edge cases, and naming consistency,
and it should suggest specific improvements with code examples. Use the git MCP
server to leave INLINE review comments on the pull request directly.
```

Claude will generate a `.md` file with two parts:

- **Frontmatter** (between `---` markers) - metadata that tells Claude *when* to use this agent
- **Body** - the system prompt that tells the agent *how* to do its job

Here's what a generated agent file looks like:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
---
name: reviewer
description: "Use this agent when the user wants to review code,
  check for issues, or get feedback on code quality.
  Trigger when the user says things like 'review my code',
  'review this PR', or 'check this code'."
---

You are a code review agent. When given code or a pull request to review, you:

1. Read the changes carefully
2. Check for common issues: error handling, edge cases, naming consistency
3. Look for anything that doesn't match the patterns used elsewhere in the
codebase
4. Suggest specific improvements with code examples
5. Use the git MCP server to post inline review comments on the pull request

Be direct. Skip anything that's fine. Only flag things worth changing.
```

Easiest option: use `/agents`. Type `/agents` in Claude Code, pick "New agent", and describe what you want. Claude creates the agent file for you interactively, and you can edit the prompt right there.

Want your agent to learn over time? Add `memory: project` to the frontmatter. The agent gets its own memory directory where it saves what it discovers across conversations - patterns, conventions, recurring issues. The story-mapping agent from Lab 3 already uses this. Review agents are a great fit because they learn your team's patterns over time.

The description is what triggers your agent. Claude reads the `description` field to decide when to automatically spawn the agent. If the description is vague, Claude won't know when to use it. Include the specific phrases a user would say, like "review my code" or "review this PR". If your agent isn't getting triggered, the description is the first thing to fix.

Iterate on the agent. After generating it, try it out. If the output isn't what you want, ask Claude to refine the agent's instructions. This back-and-forth gets you much better results than trying to write the perfect prompt on the first try.

7. Run the agent

New agent not showing up? After creating a new agent file, Claude Code needs to reload before it can use it. Either restart Claude Code, or run `/reload-plugins` to pick up the new agent without restarting.

Now just ask:

```
Review my code.
```

Watch Claude's output. You should see it spawn your `reviewer` agent as a subagent - the agent name appears in the activity stream. If Claude does the review itself instead of delegating, your agent's `description` isn't matching the prompt; refine it and try again.

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

The agent runs in its own context, reads the changes on your branch, and posts comments directly on the pull request through the git MCP server. Open the PR in your browser and confirm the comments are there.

How does Claude know to use your agent? Claude can see all agent files in `.claude/agents/`. When your prompt matches what an agent is designed to do, Claude spawns it as a subagent. You don't need a special command - just describe the task naturally.

Why a separate agent for reviews? If Claude builds code and reviews it in the same context, it's biased toward its own work. A separate review agent gets a fresh context - it only sees the code, not the reasoning that produced it. This makes it much more likely to catch real issues.

8. Optional: let Claude fix the review comments

If the review surfaced real issues, ask Claude in a new conversation:

```
Address the review comments on the PR.
```

Claude reads the comments through the git MCP, makes the fixes, commits, and pushes. Re-run "Review my code" to confirm the issues are resolved.

9. Merge the branch

Once you're happy with the PR, merge it in the browser. Then switch back to `main` locally and pull so your working copy is up to date:

```
git checkout main
git pull
```

10. Optional: run /insights

Run `/insights` in Claude Code. Claude analyzes up to 30 days of your coding sessions and generates a local HTML report. Open it and look at:

- **Bottlenecks**: where did you lose time? Repeated corrections, unclear prompts, back-and-forth that could have been avoided?
- **Recurring errors**: patterns Claude kept running into, and what that says about your CLAUDE.md or your prompting habits
- **Efficiency patterns**: what's working well that you should keep doing?

Take one suggestion from the report and act on it.

Check your work

- [ ] You created a feature branch and implemented a user story on it
- [ ] The branch is pushed and a pull request exists

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

- [ ] You created a custom review agent in `.claude/agents/`
- [ ] The agent triggered automatically on "Review my code" (it showed up as a subagent in the activity stream)
- [ ] Review comments are visible on the PR in your browser
- [ ] The PR is merged
- [ ] You ran `/insights` and acted on at least one suggestion

Reflection

- What did the review agent catch that you wouldn't have noticed yourself?
- Why does giving the review agent its own context (separate from the build) make it more useful?
- Where else in your workflow would a custom subagent be useful?
- What would you put in `memory: project` for a review agent so it gets smarter over time?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 6: Team agents - Coordinated roles

Goal

Use team agents with defined roles to implement a feature that requires coordination. By the end of this lab, your app has a complex feature built by a team of agents working together.

Context

The previous lab used subagents for independent work. This lab is different: the next spec touches multiple areas that depend on each other. You need coordination, not just parallelism.

Before you start

Team agents are an **experimental feature** in Claude Code. You need to enable them before they'll work.

If you copied the starter-pack in Lab 1, this is already configured in your `.claude/settings.json`. You can verify by checking that the file contains:

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

If it's missing, either add it to your `settings.json` or set the environment variable before launching Claude Code:

```
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
claude
```

Steps

1. Understand why this is different from subagents

Pick a spec that involves multiple concerns (e.g., data handling, UI, and code review all need to work together). Ask yourself:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

- Which parts depend on other parts being done first?
- Can any of this be done in isolation?
- What roles would help organize the work?

These pieces have dependencies and handoffs. This is a job for team agents.

Not a developer? Think of it like a real team: if you were delegating this to a handful of people with different specialties, how would you split the work? Who needs to finish before someone else can start? That's exactly the thinking you need here.

2. Plan the roles

Before launching agents, think about what roles you need. For example:

Role	Responsibility
Developer	Core logic, data handling, backend work
UX specialist	User interface, pages, styling, interactions
Developer Reviewer	Reviews the developer's code for correctness, edge cases, and consistency before handoff
Architect	Updates the ARC42 architecture docs in `docs/architecture/` to reflect the new feature

Adapt the roles to what the spec actually requires.

3. Set up and run team agents

Ask Claude to work with team coordination:

```
Implement the next spec. This requires coordination across multiple concerns.

**[Role 1] work:** [describe responsibilities]

**[Role 2] work:** [describe responsibilities]

**[Role 3] work:** [describe responsibilities]

Coordinate these as a team - [describe dependencies, e.g., "the core logic must
work before the UI can use it"].

You MUST use team agents (not workflows, not parallel subagents), but the agent
team tools.
```

4. Observe the coordination

Watch how Claude handles this:

- Does it sequence the work correctly?
- How does it handle the handoff between roles?
- Does it run tests after each stage or all at the end?
- Are there any conflicts or missed dependencies?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

5. Test the full flow

Walk through the feature manually. Does everything work end-to-end?

6. Compare approaches

Think about (or discuss with your neighbor):

- How would this have gone with a **single agent** doing everything sequentially?
- How would it have gone with **subagents** (no coordination)?
- What did team coordination give you that the other approaches wouldn't?

7. Check usage

Run `/usage`. Compare with previous labs. Team agents add overhead for coordination - was it worth it?

Check your work

- [ ] The spec is fully implemented
- [ ] All parts work together end-to-end
- [ ] All tests pass (old and new)

Reflection

- Where did coordination matter most?
- Did the agent sequence the work correctly, or did you have to intervene?
- When would you use team agents in your own work?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

Lab 7 (optional): Testing with Playwright MCP

Prerequisites

This lab only makes sense if your project has a UI that runs in a browser. Skip it for CLI tools, libraries, or API-only projects.

Goal

Let Claude drive your running app in a real browser through the Playwright MCP, explore it on its own, and then generate the Playwright E2E tests that lock in what it discovered. By the end of this lab, you have a runnable JavaScript test suite you didn't write by hand.

Steps

1. Add the Playwright MCP server

Add this entry to your `.mcp.json` at the project root (merge it with the servers you already configured in Lab 2):

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp@latest"]
    }
  }
}
```

Run `/mcp` inside Claude Code to confirm Playwright shows up. The first call downloads the package, so the initial action may take a few extra seconds.

2. Run your app locally

Start your app the way you normally would (for example, `npm run dev`). Note the URL - usually something like `http://localhost:5173` or `http://localhost:3000`.

3. Let Claude explore the app

Replace the URL with your actual one and ask:

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support

```
Using the Playwright MCP, click around in the app running on
http://localhost:5173 and do an exploratory pass. Then I'll ask you to generate
Playwright E2E tests.
```

Watch Claude open a browser window and start interacting with your app - navigating, clicking, filling forms. Let it run.

It will stop on its own when it feels like the exploration covers enough. If it keeps going past the point you find useful, press **ESC** in Claude Code to interrupt it.

4. Ask for the E2E tests

Now ask:

```
Generate Playwright E2E tests in JavaScript that cover the main user flows you
just explored. Make sure the tests pass, and make sure (and tell me) the command
I can use to run the tests myself.
```

Claude writes the test files and tells you the command. Run it in your console and watch the suite execute - much faster than the click-through Claude did earlier, because there's no LLM in the loop anymore. The exploration was the slow part; the tests are pure Playwright.

Check your work

- [ ] Playwright MCP is registered and visible in `/mcp`
- [ ] Claude explored your app in a real browser
- [ ] At least one Playwright E2E test file is generated
- [ ] The tests pass when run from the console

Reflection

- What did Claude exercise that you wouldn't have thought to test?
- How does writing tests by exploration compare to writing them up front?
- Where else in your project could this exploratory pattern apply?

© Info Support · All Rights Reserved · This data may not be copied or distributed without the prior approval of Info Support